

JUEGO NAVE ESPACIAL EN C++

CONSOLA DE WINDOWS –
MOVIMIENTO, COLISIONES Y VIDAS

OBJETIVO DEL JUEGO

- Simular una nave que puede moverse con el teclado.
- Evitar asteroides que caen desde arriba.
- La nave tiene corazones (salud) y vidas.
- El juego termina al perder todas las vidas.



ESTRUCTURA DEL CÓDIGO

- Inclusión de librerías (stdio.h, windows.h, etc.).
- Declaración de funciones auxiliares (gotoxy, pintar_limites).
- Clase NAVE: controla la nave.
- Clase AST: representa los asteroides.
- main(): ciclo del juego y lógica principal.



Librerías utilizadas



- **stdio.h**: Entrada/salida con printf.
- **windows.h**: Control del cursor (gotoxy, Sleep).
- **conio.h**: Detección de teclas (kbhit, getch).
- **stdlib.h**: Función rand() para posiciones aleatorias.

```
#include <stdio.h>
#include <windows.h>
#include <conio.h> // Para kbhit() y getch()
#include <stdlib.h> // Para rand()
```

Función gotoxy()



- Permite mover el cursor a coordenadas (x, y).
- Usada para imprimir texto en lugares específicos.
- Base para dibujar la nave, asteroides y marco.

```
// Coloca el cursor en la posición (x,y) de la consola
void gotoxy(int x, int y){
    HANDLE hCon = GetStdHandle(STD_OUTPUT_HANDLE);
    COORD dwPos = { (SHORT)x, (SHORT)y };
    SetConsoleCursorPosition(hCon, dwPos);
}
```

Función OcultarCursor()

- Obtiene el control de la ventana donde se ejecuta el juego
- Configura el cursor con tamaño 2 y lo hace invisible
- Aplica la configuración para que el cursor no se muestre durante el juego

```
// Oculta el cursor de la consola
void OcultarCursor() {
    HANDLE hCon = GetStdHandle(STD_OUTPUT_HANDLE); //obtiene el cursor
    CONSOLE_CURSOR_INFO cci = { 2, FALSE }; //ocultas
    SetConsoleCursorInfo(hCon, &cci); //aplica los cambios
}
```

FUNCIÓN PINTAR_LIMITES()

- Dibuja un marco de juego con caracteres ASCII.
- Define el área donde la nave y los asteroides se moverán.

```
// Oculta el cursor de la consola
void OcultarCursor(){
    HANDLE hCon = GetStdHandle(STD_OUTPUT_HANDLE); //obtiene el cursor
    CONSOLE_CURSOR_INFO cci = { 2, FALSE }; //oculta
    SetConsoleCursorInfo(hCon, &cci); //aplica los cambios
}
```

CLASE NAVE



Atributos:

- x, y, corazones, vidas.

Métodos:

- pintar() / borrar() → dibuja o limpia la nave.
- mover() → controla flechas y reacciona.
- pintar_corazones() → muestra salud.
- morir() → animación de muerte y reinicio.

```
// Clase que representa la nave del jugador (plantilla)
class NAVE {
    // atributos (propiedades)
    int x, y;           // Posición de la nave
    int corazones;      // Salud actual
    int vidas;          // Vidas restantes

    // constructor (sirve para darle valores a los objetos creados)
public:
    NAVE(int _x, int _y, int _corazones, int _vidas)
        : x(_x), y(_y), corazones(_corazones), vidas(_vidas) {}

    int X(){ return x; }
    int Y(){ return y; }

    // metodos (funcionalidades que el objeto puedo hacer)
    void COR() { corazones--; }
    void pintar();
    void borrar();
    void mover();
    void pintar_corazones();
    void morir();
};
```

FUNCION PINTAR()

```
// Dibuja la nave con símbolos ASCII
void NAVE::pintar() {
    gotoxy(x, y);      printf("  ^");
    gotoxy(x, y+1);    printf(" /X\\");
    gotoxy(x, y+2);    printf("/_  \\_");
}
```

FUNCION BORRAR()

```
// Borra la nave de la consola
void NAVE::borrar() {
    gotoxy(x, y);      printf("      ");
    gotoxy(x, y+1);    printf("      ");
    gotoxy(x, y+2);    printf("      ");
}
```

FUNCION MOVER()

```
// Maneja el movimiento de la nave según las teclas presionadas
void NAVE::mover() {
    if(kbhit()){ // si hay una tecla presionada
        char tecla = getch(); // Captura la tecla
        borrar();
        if(tecla == IZQUIERDA && x > 3) x--;
        if(tecla == DERECHA && x + 6 < 77) x++;
        if(tecla == ARRIBA && y > 4) y--;
        if(tecla == ABAJO && y + 3 < 23) y++;
        if(tecla == 'e') corazones--; // autolesión para prueba
        pintar();
        pintar_corazones();
    }
}
```

FUNCIÓN PINTAR_CORAZONES()

```
// Dibuja los corazones y las vidas en la parte superior de la pantalla
void NAVE::pintar_corazones() {
    gotoxy(50,2); printf("VIDAS %d", vidas); //imprimir variables
    gotoxy(64,2); printf("Salud");
    gotoxy(70,2); printf(" "); // Limpia zona
    for(int i = 0; i < corazones; i++){
        gotoxy(70 + i, 2); printf("*");
    }
}
```

FUNCIÓN MORIR()

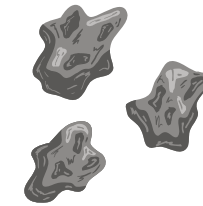
```
// Muestra animación de muerte y reinicia la nave si se queda sin corazones
void NAVE::morir() {
    if(corazones == 0) {
        borrar();
        gotoxy(x,y); printf("  **  ");
        gotoxy(x,y+1); printf("  ****  ");
        gotoxy(x,y+2); printf("  **  ");
        Sleep(200); //milisegundos

        borrar();
        gotoxy(x,y); printf(" * ** *");
        gotoxy(x,y+1); printf("  ****  ");
        gotoxy(x,y+2); printf(" * ** *");
        Sleep(200);
        borrar();

        vidas--;
        corazones = 3;
        pintar_corazones();
        pintar();

        if(vidas == 0) {
            gotoxy(30,12);
            printf("  GAME OVER  ");
            Sleep(1500);
            exit(0); // Termina el programa
        }
    }
}
```

CLASE AST



Atributos:

- posición x, y.

Métodos:

- pintar() → muestra el asteroide.
- mover() → lo hace bajar y reposiciona.
- choque(NAVE &) → detecta colisión y reduce corazones.

```
// Clase que representa un asteroide
class AST {
    int x, y; // Posición del asteroide
public:
    AST(int _x, int _y): x(_x), y(_y) {}

    void pintar();
    void mover();
    void choque(NAVE &N); // Requiere referencia a la nave
};
```

FUNCION AST::PINTAR()

```
// Dibuja el asteroide en pantalla
void AST::pintar() {
    gotoxy(x,y); printf("%c",184);
}
```

FUNCION AST::MOVER()

```
// Mueve el asteroide una fila hacia abajo
// y lo reposiciona si sale del área
void AST::mover() {
    gotoxy(x,y); printf(" ");
    y++;
    if(y > 22) {
        x = rand() % 71 + 4;
        y = 4;
    }
    pintar();
}
```

FUNCION AST::CHOQUE(NAVE &N)

```
// Detecta colisión entre asteroide y nave
void AST::choque(NAVE &N) {
    if(x >= N.X() && x < N.X()+6 && y >= N.Y() && y <= N.Y()+2) {
        N.COR();
        N.borrar();
        N.pintar();
        N.pintar_corazones();
        x = rand() % 71 + 4;
        y = 4;
    }
}
```

ANIMACIÓN DE MUERTE Y FIN DEL JUEGO

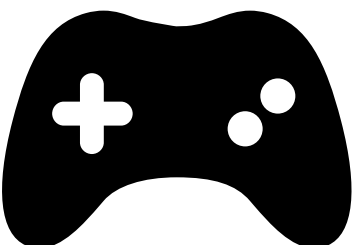
- Al quedarse sin corazones:
 - Se muestra una explosión con símbolos.
 - Se resta una vida y se reinician corazones.
- Si las vidas llegan a 0:
 - Se imprime "GAME OVER".
 - El programa se cierra con `exit(0)`.

```
// Muestra animación de muerte y reinicia la nave si se queda sin corazones
void NAVE::morir() {
    if(corazones == 0){
        borrar();
        gotoxy(x,y); printf("  **  ");
        gotoxy(x,y+1); printf("  ****  ");
        gotoxy(x,y+2); printf("  **  ");
        Sleep(200); //milisegundos

        borrar();
        gotoxy(x,y); printf(" * ** *");
        gotoxy(x,y+1); printf("  ****  ");
        gotoxy(x,y+2); printf(" * ** *");
        Sleep(200);
        borrar();

        vidas--;
        corazones = 3;
        pintar_corazones();
        pintar();

        if(vidas == 0){
            gotoxy(30,12);
            printf("  GAME OVER  ");
            Sleep(1500);
            exit(0); // Termina el programa
        }
    }
}
```



CICLO PRINCIPAL DEL JUEGO (MAIN)

1. Oculta el cursor.
2. Dibuja los límites.
3. Crea nave y asteroides.
4. Inicia un bucle infinito:
 - Mueve asteroides y verifica colisiones.
 - Llama a NAVE::morir() si fue golpeada.
 - Escucha teclas para mover la nave.
 - Espera 30 ms para controlar velocidad.

```
// Función principal del juego
int main() {
    OcultarCursor();      // Oculta cursor de texto
    pintar_limites();     // Dibuja bordes del juego

    NAVE N(7,7,3,3);      // Instancia la nave del jugador
    N.pintar();           // Dibuja la nave
    N.pintar_corazones(); // Muestra HUD de corazones y vidas

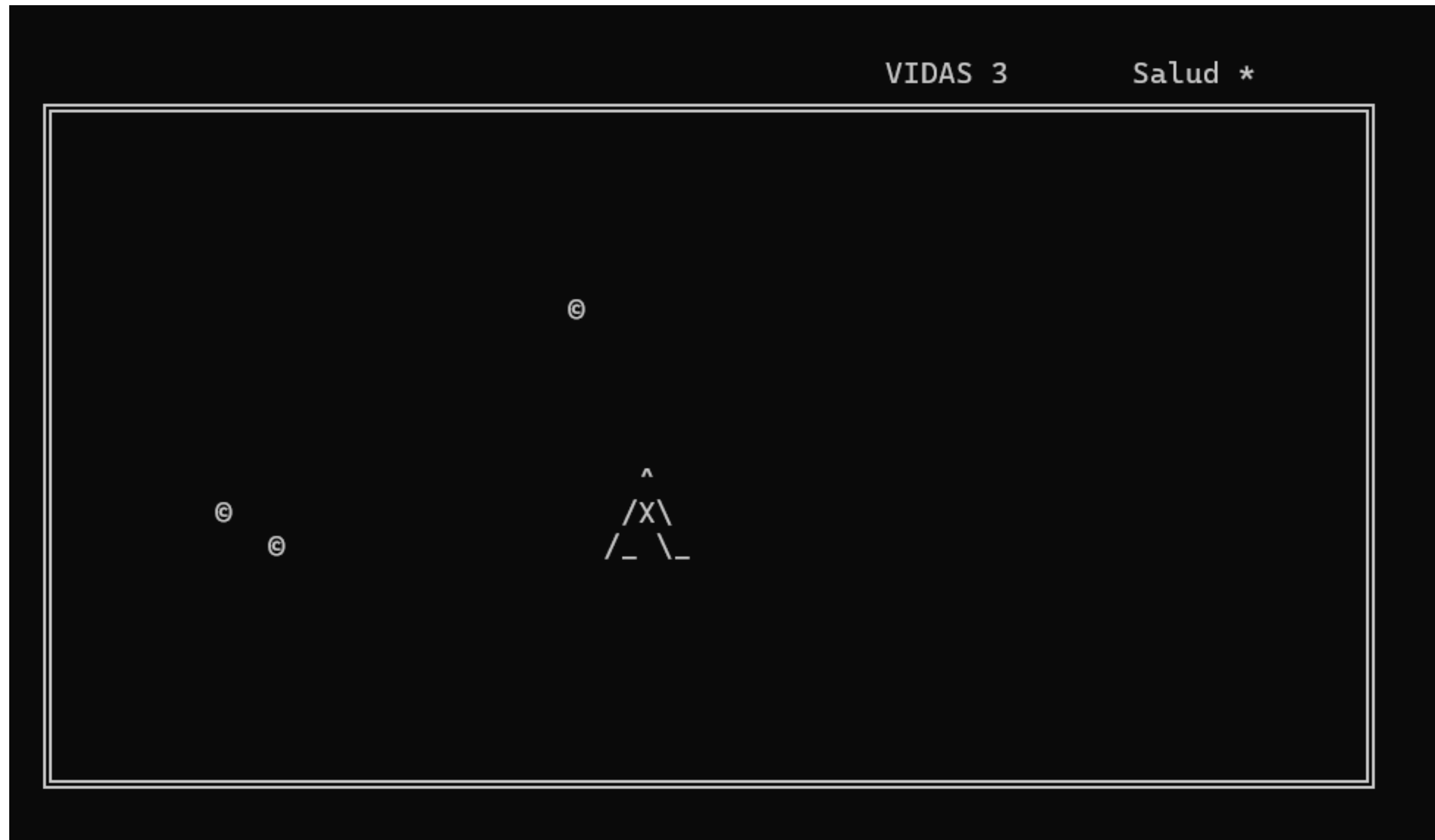
    // Instancia tres asteroides en posiciones iniciales
    AST ast1(10,4), ast2(4,8), ast3(15,10);

    // Bucle principal del juego
    while(true){
        ast1.mover(); ast1.choque(N);
        ast2.mover(); ast2.choque(N);
        ast3.mover(); ast3.choque(N);

        N.morir(); // Verifica si la nave debe morir
        N.mover(); // Mueve la nave según teclas
        Sleep(30); // Pausa para estabilidad
    }

    return 0;
}
```

JUEGO EN EJECUCIÓN



GRACIAS